

Scripts - Command Line Entry Points

Evidence producers, database guards, local helpers and the evaluation matrix

EdgePilot AI - Team 15

Documentation set, 26 August 2026

6 documents in this volume

scripts/ - command-line entry points

Work package: shared, by subfolder.

What this folder is

Everything invoked from `package.json` that is not `next` or `prisma` itself. The dividing line against `src/` is deliberate: nothing here is imported by the application, and nothing here runs in a request. These are operator tools - they produce evidence, guard a shared database, or set up local state.

What happened here

Three of the five subfolders are new. `db/` exists because a command aimed at the shared database could silently run against `localhost` and report success. `evaluation/` exists because the ten-case matrix the checklist asks for had no home. `dev/` exists because the scaffold endpoints did not persist, so rows had to be created from outside - and it survives now that they do, because driving a run from `curl` still needs rows.

Subfolders

Subfolder	What is in it
<code>benchmark/</code>	Real provider runs, failure captures, the clean-start log
<code>db/</code>	The guards around the shared Neon database
<code>dev/</code>	Local helpers for seeding and inspecting rows
<code>evaluation/</code>	The ten-case evaluation matrix
<code>vision-benchmark/</code>	Fixture generation, dataset validation, live and controlled runs

Connected folders

- `evidence/` - almost everything here writes into it.
- `/` - `package.json` is where these are named.
- `src/modules/` - the code these scripts drive.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/app/evidence/`

scripts/benchmark/ - evidence producers

Work package: benchmark and provider layer.

What this folder is

Three scripts that produce the contents of evidence/benchmark/. Between them they are the only thing in the repository that generates measured performance figures.

What happened here

run-benchmark.ts was updated alongside the runner changes - the cold-start iteration, the residency probe and the new readiness inputs all flow through it - but its contract did not change: every figure it writes carries the provenance label the adapter attached, and it deliberately does not touch the database, so it can be run before Prisma is set up.

Files

File	What it is
run-benchmark.ts	220 lines. <code>npm run bench:run -- --provider=ollama --model=... --iterations=5</code> . The only script that produces measured performance figures. Needs a provider that actually answers - a running Ollama or an API key
capture-failure-evidence.ts	578 lines. <code>npm run bench:evidence:failures</code> . Drives timeout, fallback and error-classification scenarios through the real adapters and the real runner and writes <code>failure-modes.json</code> . Exists separately from the tests because a green suite proves the behaviour to whoever runs it; an artefact proves it to whoever reads the pull request
capture-clean-start.ts	585 lines. <code>npm run bench:clean-start</code> . Runs the documented clean-environment sequence and writes down what happened, with real exit codes and real stderr. Does not summarise, does not retry silently, and exits non-zero on a failed step

Connected folders

- evidence/benchmark/ - what these write.
- src/modules/benchmark/ - the module they drive.
- docs/benchmark/ - the research log that cites their output.

Referenced from

These folders point here. Each link below resolves in both directions.

- scripts/

scripts/db/ - shared-database guards

Work package: DevOps.

What this folder is

Two files whose entire purpose is to stop a command aimed at the shared Neon database from quietly running somewhere else.

What happened here

This folder is new, and it exists because of a specific failure that is easy to miss and expensive to discover late.

`dotenv -e .env.neon -- prisma migrate deploy` fails **silently** when `.env.neon` is absent. `dotenv-cli` loads nothing, Prisma falls back to its own `.env`, and a command named "neon" runs against localhost while printing "No pending migrations to apply." Nothing about the output says the target was wrong.

The inverse is worse: a file that is present but points somewhere unexpected lets a command that looks local reach a shared database, and migrate against the wrong database is not something you can recover from by reading the output afterwards.

A second, smaller problem: flags did not survive the trip. `npx dotenv -e f -- prisma migrate resolve --applied X loses --applied` somewhere between `npx`, `dotenv-cli` and the shell, and Prisma then complains about a flag you clearly passed.

Files

File	What it is
<code>neon.mjs</code>	126 lines. The single entry point for Prisma commands against the shared database. Reads the env file itself, sets the variables, prints the destination host before doing anything , and passes flags through intact. Backs <code>db:neon</code> , <code>db:neon:deploy</code> , <code>db:neon:seed</code> , <code>db:neon:studio</code> , <code>db:neon:status</code>
<code>require-env-file.mjs</code>	71 lines. Refuses to proceed unless <code>.env.neon</code> exists, parses, and names a host that is not local

The rule these enforce

Against the shared database, only `migrate deploy`. Never `migrate dev` - it can offer to reset, and a reset wipes everyone's data.

Connected folders

- `prisma/` and `prisma/migrations/` - what these commands apply.
- `docs/internal/` - `database.md` describes the local-versus-shared split.
- `/` - `.env.example` documents the variables; `.env.neon` itself is never committed.

Referenced from

These folders point here. Each link below resolves in both directions.

- `scripts/`

scripts/dev/ - local development helpers

Work package: benchmark and DevOps.

What this folder is

Two throwaway-grade helpers for working against a local database. Neither is imported by the application and neither runs in CI.

What happened here

Both files broke next build during the device removal, and that is the most useful thing about them. `show-last-run.ts` read `run.deviceId` and `seed-demo-rows.ts` called `prisma.device.create`. Jest never imports either file, so the test suite stayed green while the project would not compile. This is the concrete reason `npm run build` is a required check and not an optional one.

`seed-demo-rows.ts` is now mostly obsolete by design. It existed because the workload and device endpoints were scaffold stubs that echoed `temp-*id` and never wrote anything, so an end-to-end run had to have its rows seeded from outside and their uuids pasted into the dashboard by hand. Both of those problems are gone. It survives for driving a run from curl, where there is no browser to generate a session id.

Files

File	What it is
<code>seed-demo-rows.ts</code>	75 lines. Creates the rows a benchmark run needs, for use outside the browser
<code>show-last-run.ts</code>	104 lines. Prints what the last benchmark run actually persisted, read straight from the database. The dashboard reporting persisted: <code>true</code> is the API telling you what it believes; this is independent evidence the rows are really there

Connected folders

- `prisma/` - the schema both read.
- `src/app/api/v1/workloads/` - the endpoint that made the seeding helper necessary, and then unnecessary.

Referenced from

These folders point here. Each link below resolves in both directions.

- `scripts/`

scripts/evaluation/ - the ten-case matrix

Work package: benchmark and provider layer.

What this folder is

One script producing one artefact: the reproducible ten-case evaluation the submission checklist asks for.

What happened here

New. The behaviours were already covered by unit tests, but scattered - a reviewer had to go and find them. This runs them in one place, writes a single readable artefact, and exits non-zero if any case misbehaves, so it can sit in a pipeline.

What it proves is bounded and stated in the artefact itself: every case drives **real module code** - the same zod schemas, the same normaliser, the same readiness calculator the application uses - so the cases cannot pass against a mock that has drifted. No network call is made and no result is transcribed by hand.

Files

File	What it is
ten-case-matrix.ts	340 lines. <code>npm run eval:matrix</code> . Ten cases across six categories: 1 normal, 3 malformed, 1 ambiguous, 3 injection, 1 missing-evidence, 1 provider-failure. Case 9 asserts that a missing hardware fit stays null and never becomes 0

On the injection cases

EdgePilot never *acts* on model output - it does not execute it, follow it, or pass it to another system. The risk is narrower and easier to miss: output being read as a **verdict**. So the defence is exact matching against a closed label set, with anything that does not match recorded as `invalid_output` rather than guessed at. The threat model is written into the artefact so it can be argued with.

Connected folders

- `evidence/evaluation/` - the artefact.
- `src/modules/benchmark/core/services/` - the readiness calculator and assessors under test.
- `src/modules/vision-benchmark/core/` - the normaliser the injection cases drive.

Referenced from

These folders point here. Each link below resolves in both directions.

- `scripts/`
- `src/app/evaluation/`
- `tests/evaluation/`

scripts/vision-benchmark/ - vision dataset and run tooling

Work package: vision benchmark.

What this folder is

Four scripts: one that builds the dataset, one that checks it, and two that run a benchmark against it - one controlled, one live.

What happened here

run.ts and validate-dataset.ts both changed to support datasets that are not the built-in seven-class one. validate-dataset.ts grew from 57 to 75 lines, mostly to stop a specific false failure: perClass was pinned to .length(VISION_LABELS.length), so a dataset with a different number of classes would run to completion and then be rejected by validation. Six fields had to be widened together, TypeScript interfaces and zod schemas in the same pass - widening one without the other cost three separate build round-trips.

Two operational traps are worth recording because they cost real time: npm run ... -- --flag does not always forward the flag through to the script, and node --import tsx does not load .env, so OLLAMA_VISION_MODEL was invisible and the script silently defaulted to a model that was not installed. Pass the model explicitly.

Files

File	What it is
generate-fixtures.mjs	727 lines. npm run vision:fixtures. Builds the dataset images deterministically. Called automatically by prebuild and pretest, so a build or a test run never starts from a missing dataset
validate-dataset.ts	76 lines. npm run vision:validate. Checks the manifest against the files on disk
run.ts	114 lines. npm run vision:run:ollama / vision:run:gemini / vision:run:groq. A live run against a real model, writing to evidence/vision-benchmark/
generate-controlled-evidence.ts	144 lines. npm run vision:evidence:controlled. Deterministic runs against scripted providers - proves the pipeline, not the model

Connected folders

- datasets/vision-benchmark/ - what these build and validate.
- evidence/vision-benchmark/ - what the two run scripts write.
- src/modules/vision-benchmark/ - the module they drive.
- docs/vision-benchmark/ - provider-execution.md documents these commands.

Referenced from

These folders point here. Each link below resolves in both directions.

- .github/workflows/
- datasets/
- scripts/